



PR204

Réseaux et Système - 2024/2025

ENSEIRB-MATMECA
TELECOMMUNICATIONS

Rédigé par
MONTI Vincent
WASSATE Yassine

Encadrants :
G. MERCIER
J. BRUNEAU-QUEYREIX
P. SWARTVAGHER

Table des matières

1	Introduction	1
2	Phase 1	1
2.1	dsmexec	1
2.2	dsmwrap	4
3	Phase 2	5
3.1	dsm_init	5
3.2	segv_handler	6
3.3	dsm_listener_thread	6
3.4	dsm_handler	6
3.5	dsm_finalize	7
3.6	exemple.c	7
4	Conclusion	8

1 Introduction

Le but de ce projet est de construire un système de mémoire partagée sur des ordinateurs distants ayant la même architecture. Le projet comporte deux phases et nous les aborderons dans l'ordre.

2 Phase 1

2.1 dsmexec

Tout d'abord, nous commençons par construire la fonction **dsmexec**. Cette fonction est la première fonction exécutée qui permet d'initier l'ensemble de l'architecture du projet. La première action effectuée est d'assigner un traitant aux signaux **SIGCHLD** afin que les processus enfants se terminent correctement. Immédiatement après, nous extrayons l'information du fichier **machine_file**. Ce fichier contient les noms des machines avec lesquelles nous souhaitons construire la structure de mémoire partagée. Rentrons dans le détail de son organisation.

La fonction **read_machine_file** prend le nom du fichier où sont stockés les noms des machines et renvoie un tableau en sortie. Le premier élément du tableau est le nombre de machines et les cases suivantes contiennent leurs noms. La fonction réalise une première lecture du fichier pour compter les lignes non vides et allouer la mémoire du tableau. Ensuite, la lecture est reprise du début pour remplir les cases du tableau par les noms des machines. Enfin, le nombre de machines repertoriées est placé dans la première case. Le procédé est robuste et permet de ne pas prendre en compte d'éventuelles lignes vides au milieu de la liste des noms. Poursuivons dans la construction de **dsmexec**.

Le nombre de machines impliquées étant connu, nous pouvons allouer la mémoire pour la structure d'information de ces machines. Puis, l'adresse IP de la machine locale est obtenue grâce à la fonction **get_local_ip**. Cette dernière ne prend rien en argument et renvoie l'IP sous forme de chaîne de caractères. La récupération de l'adresse passe par l'utilisation du module **netdb.h** et plus particulièrement de la fonction **gethostname** renvoyant un pointeur sur une structure **hostent** contenant, entre autres, l'adresse IP de l'hôte local. Une fois l'adresse IP locale obtenue, il devient possible d'initialiser la socket d'écoute du processus principal.

Cette opération est accomplie par la fonction **creer_socket**. Elle nécessite en entrée le type de socket souhaitée, l'adresse IP locale et un port pour nous renvoyer le descripteur de fichier de la socket créée. En laissant le port à 0, ce dernier sera choisi aléatoirement. Egalement, la fonction paramètre la socket et la place en écoute avec jusqu'à possiblement 10 connexions en file d'attente. Le nombre de machines employées ne devant pas dépasser 4, ce nombre de connexions est une sécurité supplémentaire. Le port étant choisi aléatoirement dans notre cas, nous le récupérons directement après grâce à la fonction **getsockname** du module **sys/socket.h**.

Ces actions effectuées, nous devons maintenant créer autant de processus enfants qu'il y a de machines impliquées dans notre architecture de mémoire partagée. Souhaitant les synchroniser au moment de leur échange d'informations avec le processus parent, nous mettons en place une barrière de synchronisation. Toutefois, cette dernière a été mal employée et n'a pas rempli le rôle souhaité. Nous nous intéressons désormais à la boucle de séparation des processus.

Pour commencer, il nous faut rediriger les messages des processus enfants vers le processus parent. Lorsque ces processus se termineront pour commencer l'exécution du programme **dsmwrap** sur les machines distantes, le protocole **ssh** transmettra les messages vers la machine locale. L'objectif est donc de faire le lien entre la transmission par **ssh** et le processus parent. Ce lien sera fait par un tube pour chaque sortie (d'écriture standard, et d'erreur standard). Tous les processus écriront dans un couple de tubes chacun. Nous créons donc les tubes par la fonction **pipe** du module **unistd.h**. Puis, le processus parent crée les processus enfant par la fonction **fork** du même module. Cette dernière renvoyant un PID égal à 0 si dans le processus enfant et le PID du processus enfant créé si dans le processus parent, nous pouvons les différencier dans le flot d'exécution.

Intéressons nous aux processus enfants dans un premier temps. Dès l'entrée dans ce processus, nous fermons la sortie en écriture standard. Puis, nous dupliquons l'extrémité en écriture du tube associé et répétons l'opération pour la sortie d'erreur standard. La duplication choisissant le premier descripteur de fichier disponible, il est certain que le descripteur choisi sera celui précédemment utilisé par la sortie que nous voulons rediriger. Suite à cela, nous créons la ligne de commande qui sera utilisée par la fonction **execvp** du module **unistd.h**. Cette dernière arrête immédiatement le processus appelant pour exécuter la ligne de commande donnée en argument. Notre ligne de commande est l'utilisation du protocole **ssh** pour se connecter à la machine distante.

Passons au processus parent. Dans la boucle, ce dernier se contente de fermer les extrémités inutiles du tube avant de mettre à jour la structure d'information des processus. Après la boucle, le parent en commence une autre se mettant en attente de connexion de tous les processus distants pour les accepter. Il reçoit ensuite leurs informations de connexion res-

pectives pour savoir comment contacter les machines distantes. Ensuite, le processus parent transmet à chaque processus le nombre total de processus, leur place et les informations de connexion de chacun. En suivant, il se met en écoute des processus distants pour afficher les messages reçus dans la console. Enfin, lorsque les processus distants seront terminés, il ferme la socket d'écoute et libère la mémoire de l'ensemble des structures d'information et tableaux utilisés.

Lorsque chaque processus enfant appelle la fonction **execvp**, il exécute non seulement la commande **ssh** pour se connecter à la machine distante mais demande à cette dernière commande d'exécuter le code de **dsmwrap**. Penchons nous sur son organisation.

2.2 dsmwrap

Pour commencer, **dsmwrap** vérifie qu'il reçoit le bon nombre d'argument et provoque l'arrêt immédiat de l'exécution dans le cas contraire. Ensuite, il crée la socket de communication avec le processus initial et s'y connecte pour transmettre les informations de sa machine. Puis, il utilise la fonction **creer_socket** abordée précédemment pour initialiser la socket d'écoute en vue de la communication avec les autres processus. Suite à cela, il duplique les descripteurs de fichier de sa socket réservée avec le processus principal et de sa socket d'écoute pour permettre d'y accéder à posteriori. Enfin, il crée la ligne de commande qu'il fait exécuter par la fonction **execvp** faisant appel au programme transmis dès le départ par **dsmexec**.

La fonction dswrap se termine après l'appel à **execvp** et laisse place au programme de notre choix. Toutefois, la fonction **dsm_init** est une étape nécessaire à la construction de notre architecture de mémoire partagée. La construction de cette dernière est l'objet de la **Phase2** de notre projet.

3 Phase 2

3.1 `dsm_init`

Après l'appel à `execvp`, le programme `exemple.c` est exécuté. Il s'agit d'un programme de test de la dsm. La première action effectuée est d'appeler `dsm_init`.

Cette fonction a pour but de mettre en place les connexions entre les processus. Elle commence par récupérer les descripteurs de fichiers des sockets utiles sous forme de variables d'environnement. Ensuite, `dsmexec` lui transmet des informations : le nombre total de processus, son rang et les informations de connexion des autres processus. `dsm_init` peut alors commencer son travail de mise en place des connexions entre processus de dsm. Pour cela, un appel à la fonction intermédiaire `dsm_comm_daemon` est nécessaire.

Le daemon place la socket `MASTER_FD`, assignée au processus local, en écoute. Ensuite, l'action dépend du `DSM_NODE_ID`. S'il existe des processus dont l'ID est inférieur à celui du processus local, ce dernier tente de s'y connecter puis envoie son ID pour être identifié. A contrario, s'il existe des processus dont l'ID est supérieur à celui du processus local, ce dernier accepte leur connexion et récupère leur ID. Une fois fait, le daemon se termine.

Poursuivons dans `dsm_init`. La mise en place des connexions étant effectuée, nous voulons les tester. Nous initialisons alors une structure de test qui servira de message. Elle est composée d'un type arbitraire, de l'ID de la source et d'un message défini. Tous les processus envoient la structure à tous les autres et attendent de la récupérer. Cela permet de tester l'ensemble des connexions dans les deux sens. Une fois la garantie que les connexions sont fonctionnelles, les pages sont allouées et un traitant est assigné au signal `SIGSEGV`. Ce dernier se nomme `segv_handler`.

3.2 `segv_handler`

Le traitant `segv_handler` dépend de l'adresse de l'erreur. Si cette adresse est comprise dans les pages de l'architecture de dsm, il s'agit d'une demande d'accès à une page. Un thread débute alors pour générer la structure de la requête et l'envoyer au processus détectant la page. Dans le cas contraire, l'erreur de segmentation est une erreur "normale" et relayée comme telle. Une fois ce traitant mis en place, un thread d'écoute est déclenché par `dsm_init`. Ce dernier se nomme `dsm_listener_thread`.

3.3 `dsm_listener_thread`

Le thread d'écoute crée un tableau de descripteur de fichier correspondant aux sockets de communications avec les autres processus. Puis, il attend une activité quelconque sur toutes les sockets. Dès qu'une activité est détectée, il fait appel à la fonction `dsm_handler` afin de gérer les différentes requêtes de dsm. Passons à la composition de cette fonction.

3.4 `dsm_handler`

Tout d'abord, le handler commence par récupérer la structure de communication. Ensuite, un switch nous permet de traiter les demandes. S'il s'agit d'une demande de type **REQ**, il s'agit d'une demande d'accès à la page. Le handler envoie alors les informations liées à la page avant d'envoyer les données de la page. Puis, il envoie une demande de type **NREQ**. Cette dernière sert à informer les autres processus d'un changement de propriétaire sur une page. Si le handler reçoit cette requête, il récupère les informations liées à la page pour les mettre à jour. S'il s'agit d'une requête de type **PAGE**, cela signifie qu'une page a été demandée précédemment et que le processus va la recevoir. Il agit donc en consé-

quence en récupérant informations et données. Finalement, dans le cas d'une requête de type **FINALISE**, il s'agit d'une finalisation du processus et informe donc que le processus se retire de l'architecture. Cela se fait par le biais de la fonction **dsm_finalize**.

3.5 dsm_finalize

Nous arrivons à la dernière fonction. **dsm_finalize** est là pour arrêter l'action d'un des processus dans l'architecture de la dsm. Cette fonction crée une structure de finalisation et l'envoie à tous les processus. Puis, elle arrête le thread d'écoute. Ensuite, elle ferme l'ensemble des socket. Par la suite, elle libère l'ensemble des tableaux et structures utilisés. Enfin, elle incrémente une variable d'environnement servant de compteur de processus finalisés. Ce compteur sert dans la fonction **exemple.c**.

3.6 exemple.c

La fonction **exemple.c**, une fois l'appel à **dsm_init** effectué, crée la variable d'environnement **FINALIZE_COUNTER** et l'initialise à 0. Elle est utilisée comme dit précédemment par **dsm_finalize**. Une boucle dans **exemple.c** permet de récupérer et comparer sa valeur. S'il ne reste qu'un seul programme en vie, cela lui permet de savoir qu'il ne doit pas utiliser **dsm_finalize** comme les autres processus mais simplement se terminer. Toutefois, cette technique a comme désavantage de ne pas libérer intégralement la mémoire.

4 Conclusion

Ce projet avait pour but de mettre en place une architecture de pages partagées par plusieurs machines. Nous avons commencé par déterminer le nombre de machines impliquées afin de créer des processus enfants correspondants. Ces derniers avaient pour but de préparer une ligne de commande pour exécuter un programme intermédiaire nettoyant la ligne de commande et se connectant avec le processus initial. Une fois cette connexion effectuée, il préparait une socket d'écoute pour le processus local et exécutait un autre programme. Ce programme lançait alors l'architecture de mémoire partagée par une fonction. Cette dernière récupérait les sockets, initialisait les communications avec les autres processus et assignait un traitant de signal aux erreurs de segmentation. Enfin, elle déclenchait un thread d'écoute des requêtes de dsm qui traitait chacune d'elle selon leur type. Lorsqu'un programme souhaitait se terminer, il employait une fonction de finalisation pour libérer la mémoire et informer les autres processus de son arrêt. Enfin, la fonction exécutée en dernier sur la machine distante comptait le nombre de processus déjà terminés pour ne pas attendre un message de finalisation inutilement.